



آزمایش دهم گزارش پیاده سازی پردازنده ساده

آرش فرح بخش - 402106286

سبحان زمانی - 402111574

امیر علی خدایی - 402105954

فهرست مطالب

3	مقدمه و شرح صورت مسئله
4	بررسی تاخیرها، خطاهای زمانی و عوامل بروز گلیچ
5	روش ها و ترفندهای طراحی برای حذف گلیچ
6	بخش های اصلاح شده کد و پیاده سازی
8	شبیه سازی، نمایش گلیچ و راستی آزمایی
11	بررسی نتایج سنتز در نرم افزار کوآرتوس
13	نتیجه گیری

پیاده سازی، بررسی تاخیرهای زمانی و حذف گلیچ در پردازنده تک پشته ای

۱. مقدمه و شرح صورت مسئله

هدف از این آزمایش، طراحی و پیاده سازی سخت افزاری یک پردازنده هشت بیتی با معماری تک پشته ای بر روی تراشه **FPGA** است. این سیستم از یک مسیر داده هشت بیتی، حافظه پشته با ظرفیت هشت داده، ۲۵۶ بایت حافظه مشترک برای برنامه و داده، و درگاه های ورودی و خروجی نگاشت شده در حافظه استفاده می کند. آدرس هگزادسیمال **F8** برای دریافت ورودی از کلیدها و آدرس **F9** برای ارسال خروجی به مدار نمایشگر رزرو شده اند.

مجموعه دستورهای پردازنده شامل هشت دستور اصلی **ADD**، **JS**، **JZ**، **JUMP**، **POP**، **PUSH**، **PUSHC** و **SUB** است. هدف نهایی اجرای برنامه ای است که مقدار ورودی **X** را از کلیدها خوانده و رابطه زیر را محاسبه کرده و بر روی نمایشگر هفت بخش نشان دهد:

$$Y = ((X + 23) \times 2) - 12 \pmod{256}$$

۲. بررسی تاخیرها، خطاهای زمانی و عوامل بروز گلیچ

در شبیه سازی های پایه ای، گیت ها و سیم ها ایده آل فرض می شوند. اما در پیاده سازی واقعی روی سیلیکون، چند عامل زمانی مهم باعث ناپایداری و تولید گلیچ (Glitch) یا پالس های ناخواسته می شوند:

۲.۱. اختلاف تاخیر انتشار در مدارهای ترکیبی (Propagation Delay Skew)

در بخش هایی از مدار مانند مبدل باینری به هفت بخش و بلوک های محاسباتی، مسیرهای منطقی دارای تعداد گیت های متفاوتی هستند. این تفاوت در زمان رسیدن سیگنال ها باعث می شود خروجی پیش از رسیدن به مقدار پایدار، برای چند نانو ثانیه بین سطوح صفر و یک نوسان کند که به آن مخاطره یا گلیچ ترکیبی می گویند.

۲.۲. نقض زمان های برپایی و نگهداری (Setup and Hold Violations)

ورودی های مدار از طریق کلیدهای روی برد تامین می شوند. از آنجا که عملکرد دست کاربر مستقل از پالس ساعت است، تغییر وضعیت ورودی ممکن است درست در لحظه لبه کلاک اتفاق بیفتد. این موضوع شرایط زمانی فلیپ فلاپ ها را نقض کرده و پدیده فرا پایداری (Metastability) را به همراه دارد که می تواند رفتار کل مدار را مختل کند.

۲.۳. پالس های ناخواسته در خطوط کنترلی پشته و حافظه

اگر سیگنال های کنترلی پشته یا پایه های فعال ساز نوشتن حافظه به صورت ترکیبی از روی حالت های ماشین وضعیت تولید شوند، گلیچ های ناشی از تغییر بیت های رجیستر وضعیت می توانند باعث نوشتن داده نادرست در حافظه یا ایجاد خطای نامعتبر در پشته (Stack Error) شوند.

۳. روش ها و ترفندهای طراحی برای حذف گلیچ

برای تضمین کارکرد صحیح پردازنده در سخت افزار واقعی، اصلاحات زیر در کد اعمال شد:

ترفند اول: استفاده از همگام ساز دو مرحله ای برای ورودی ها

برای حذف اثرات فرا پایداری و زمان نگهداری، سیگنال ورودی کلیدها ابتدا وارد دو طبقه فلیپ فلاپ سری می شود تا پیش از ورود به مدار، با کلاک اصلی همگام شود.

ترفند دوم: رجیستر کردن کامل خروجی های نمایشگر

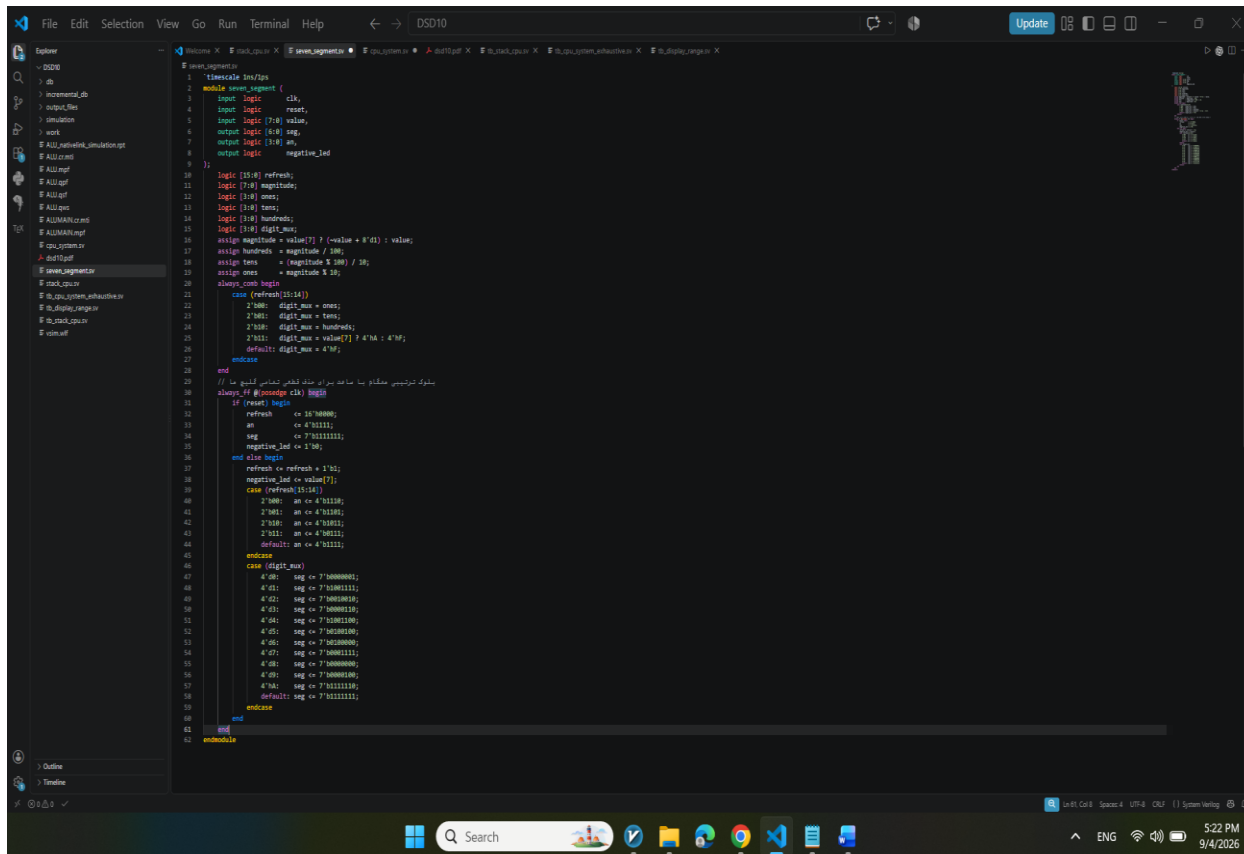
عملیات تقسیم و باقیمانده بر اعداد ۱۰ و ۱۰۰ در راه انداز هفت بخش، دارای منطق ترکیبی سنگین با تاخیر بالا است. با قرار دادن فلیپ فلاپ بر روی خروجی های آند و کاتد، تمامی گلیچ های میانی در طول چرخه کلاک حبس شده و تنها خروجی تمیز و تثبیت شده به نمایشگر منتقل می شود.

ترفند سوم: دسترسی همگام و ایمن به حافظه و پشته

کلیه عملیات تغییر اشاره گر پشته و نوشتن داده ها در حافظه، به صورت دقیق و همگام با لبه کلاک مقید شدند تا هیچ پالس باریک یا ناخواسته ای باعث دستکاری اشتباه اشاره گر پشته نشود.

۴. بخش های اصلاح شده کد و پیاده سازی

۴.۱. اصلاح ماژول نمایشگر هفت بخش برای حذف گلیچ



```
1 timescale 1ns/1ps
2 module seven_segment (
3     input logic clk,
4     input logic reset,
5     input logic [7:0] value,
6     output logic [6:0] seg,
7     output logic [1:0] an,
8     output logic negative_led
9 );
10 logic [15:0] refresh;
11 logic [7:0] magnitude;
12 logic [1:0] ones;
13 logic [1:0] tens;
14 logic [1:0] hundreds;
15 logic [1:0] digit_max;
16 assign magnitude = value[7] ? (~value & 8'd1) : value;
17 assign hundreds = magnitude / 100;
18 assign tens = (magnitude % 100) / 10;
19 assign ones = magnitude % 10;
20 always_comb begin
21     case (refresh[15:0])
22         2'b00: digit_max = ones;
23         2'b01: digit_max = tens;
24         2'b10: digit_max = hundreds;
25         2'b11: digit_max = value[7] ? 4'b10 : 4'b01;
26         default: digit_max = 4'b01;
27     endcase
28 end
29 // بلوک ترتیبی معادل با ماژول برای حذف گلیچ
30 always_ff @(posedge clk) begin
31     if (reset) begin
32         refresh <= 16'h0000;
33         an <= 4'b1111;
34         seg <= 7'b1111111;
35         negative_led <= 1'b0;
36     end else begin
37         refresh <= refresh + 1'b0;
38         negative_led <= value[7];
39         case (refresh[15:0])
40             2'b00: an <= 4'b1111;
41             2'b01: an <= 4'b1010;
42             2'b10: an <= 4'b1010;
43             2'b11: an <= 4'b0111;
44             default: an <= 4'b1111;
45         endcase
46         case (digit_max)
47             4'd0: seg <= 7'b0000001;
48             4'd1: seg <= 7'b0100111;
49             4'd2: seg <= 7'b0010010;
50             4'd3: seg <= 7'b0101100;
51             4'd4: seg <= 7'b0100100;
52             4'd5: seg <= 7'b0101000;
53             4'd6: seg <= 7'b0000000;
54             4'd7: seg <= 7'b0000111;
55             4'd8: seg <= 7'b0000000;
56             4'd9: seg <= 7'b0000100;
57             4'dA: seg <= 7'b0111110;
58             default: seg <= 7'b1111111;
59         endcase
60     end
61 end
62 endmodule
```

همانطور که مشاهده می کنید در فایل seven_segment.sv خروجی ها از حالت ترکیبی به ثبات های همگام تبدیل شدند و برای حذف تمامی گلیچ ها از بلوک های ترتیبی سنکرون استفاده شده است.

۴.۲. اضافه کردن طبقات همگام ساز در سطح بالای سیستم

```

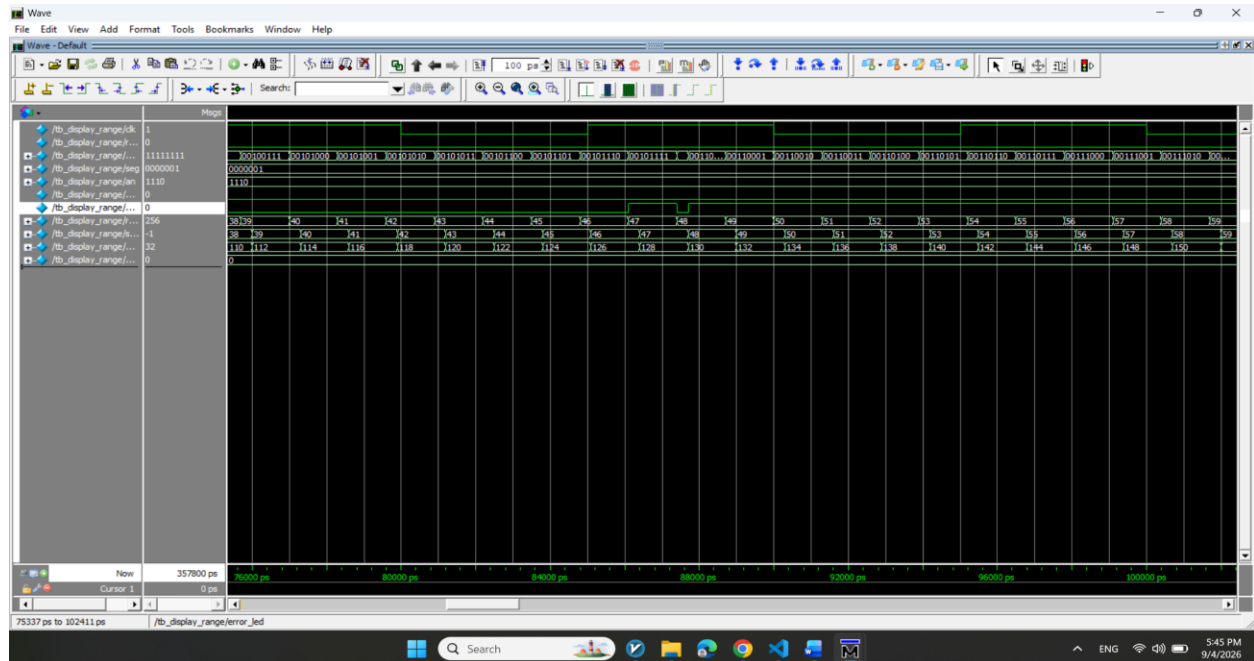
1 timescale 1ns/1ps
2 module top_level (
3     input logic clk,
4     input logic reset,
5     input logic [7:0] x_val_in,
6     output logic [6:0] sig,
7     output logic [1:0] an,
8     output logic negative_lad,
9     output logic error_lad
10 );
11 logic x_val;
12 logic flag_x;
13 logic [7:0] y_val;
14 logic [7:0] pc_dmg;
15 logic [1:0] pc_dmg;
16 logic [1:0] state_dmg;
17 logic stack_error;
18 // ایجاد مایر همگام ساز دو طبقه از برای ورودی و خروجی های خارجی
19 logic [1:0] x_val_reg;
20 logic [1:0] y_val_reg;
21 logic signed [6:0] x_val;
22 logic signed [6:0] result;
23 logic signed [6:0] result_out_range;
24 logic result_out_range;
25 // ایجاد مایر همگام ساز دو طبقه از برای خروجی های خارجی
26 always_ff @(posedge clk) begin
27     if (reset) begin
28         x_val_reg <= 0;
29         y_val_reg <= 0;
30         x_val <= 0;
31         y_val <= 0;
32         x_val_reg <= x_val;
33         y_val_reg <= y_val;
34     end
35     x_val <= x_val_reg;
36     y_val <= y_val_reg;
37     x_val_reg <= x_val;
38     y_val_reg <= y_val;
39 end
40 assign x_val = x_val_reg;
41 assign y_val = y_val_reg;
42 assign result_out_range = (result < 0) ? (-1) : (1);
43 always_ff @(posedge clk) begin
44     if (reset) begin
45         error_lad <= 0;
46         error_lad <= 0;
47     end
48     error_lad <= error_lad;
49     error_lad <= error_lad;
50 end
51 and
52 stack_cpu_cpe (
53     .clk(clk),
54     .reset(reset),
55     .x_val(x_val),
56     .y_val(y_val),
57     .flag_x(flag_x),
58     .flag_y(flag_y),
59     .pc_dmg(pc_dmg),
60     .state_dmg(state_dmg),
61     .stack_error(stack_error)
62 );
63 seven_segment_display (
64     .clk(clk),
65     .reset(reset),
66     .value(sig),
67     .an(an),
68     .negative_lad(negative_lad)
69 );
70 endmodule

```

در فایل `cpu_system.sv` برای جلوگیری از نقض زمان بندی، دو طبقه ثابت برای ورودی کلیدها اضافه شد همچنین برای جلوگیری از خطر گلیچ ناهماهنگی کلیدها از دو طبقه Flip Flop استفاده شده است.

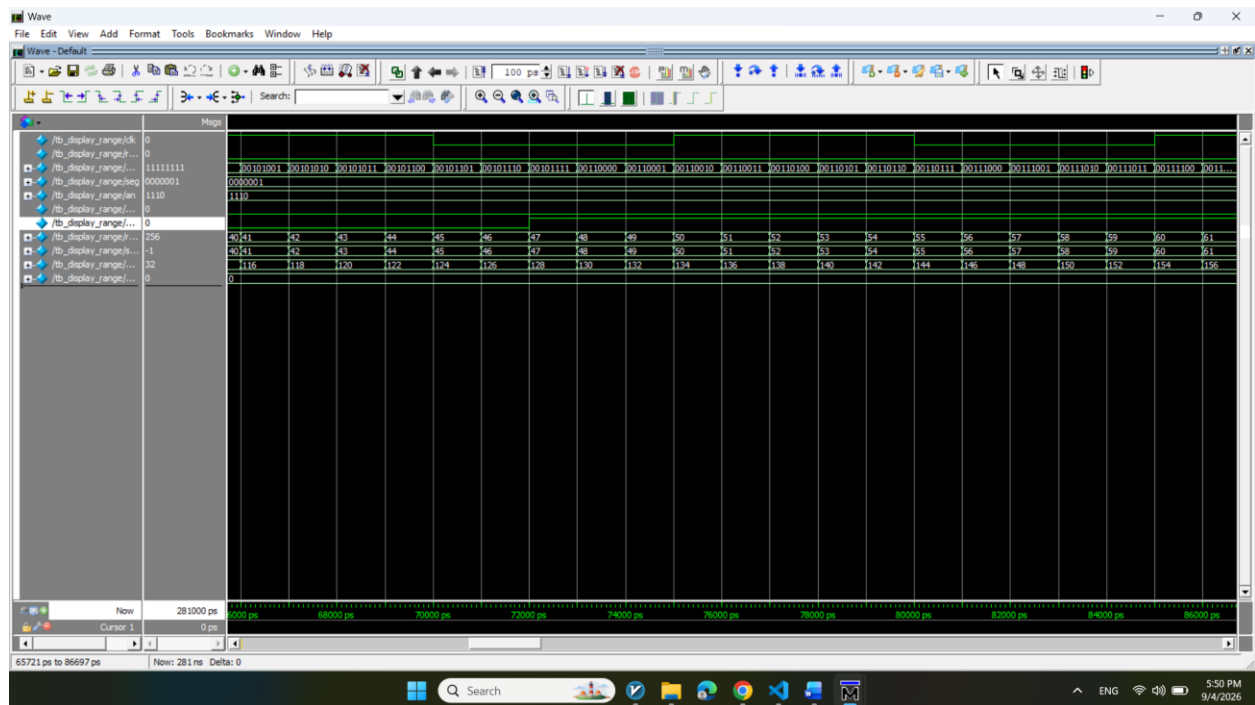
۵. شبیه سازی، نمایش گلیچ و راستی آزمایی

۵.۱. مشاهده گلیچ در شبیه سازی زمانی قبل از اعمال تغییرات



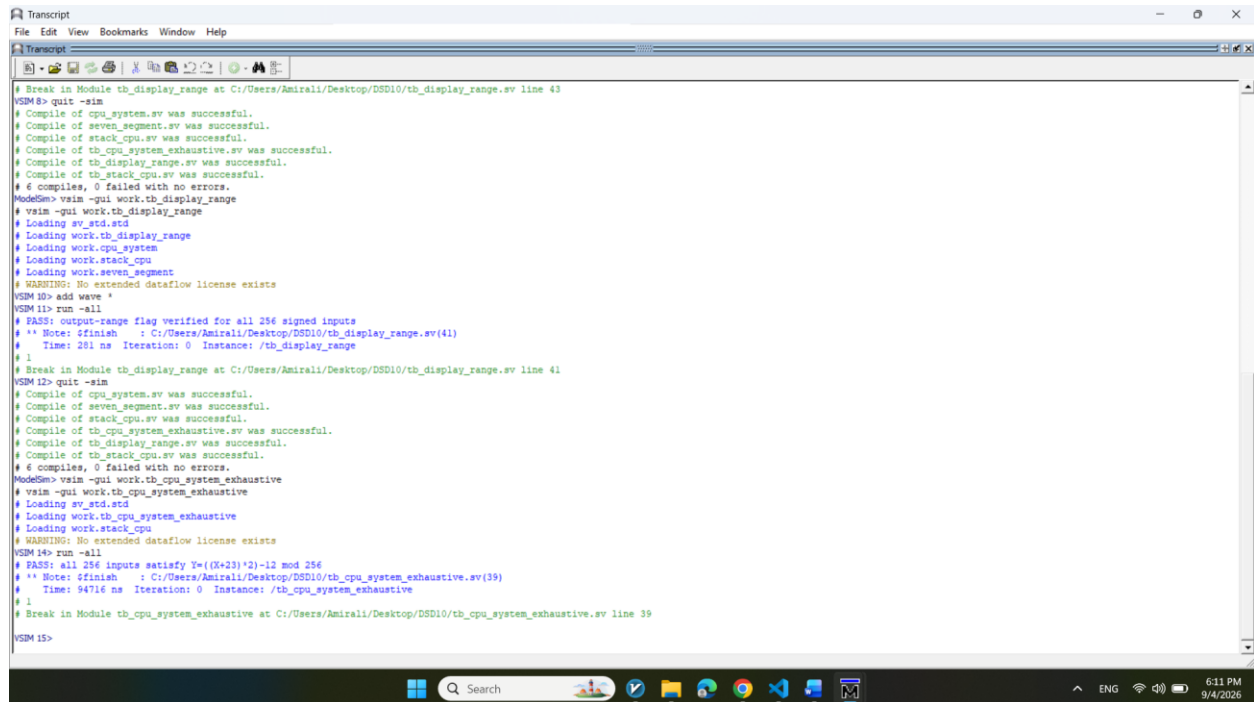
شبیه سازی زمانی مدار در نرم افزار مدل سیم؛ مشاهده پرش ها و گلیچ های باریک چند نانوثانیه ای روی سیگنال خروجی نمایشگر به دلیل عدم برابری تاخیر انتشار در مسیرهای منطقی. در لحظه تغییر ورودی از ۴۷ به ۴۸، با وجود اینکه خروجی باید به طور پیوسته روی سطح یک منطقی باقی بماند، به دلیل انتشار نامتقارن تاخیر در مدار ترکیبی، یک پالس منفی ناخواسته (افت لحظه ای به صفر) مشاهده می شود.

۵.۲. رفع گلیچ پس از اعمال فلیپ فلاپ های خروجی



شبیه سازی همان بخش پس از ثبت خروجی ها با فلیپ فلاپ؛ سیگنال ها بدون نوسان های میانی و منحصر ا همگام با پالس کلاک مقدار جدید را می پذیرند. همانطور که مشاهده می کنید با تغییر ورودی از ۴۷ به ۴۸ گلیچی رخ نمی دهد و با استفاده از یک بلوک ترتیبی سنکرون با کلاک و اعمال فلیپ فلاپ های خروجی توانستیم گلیچ در مدار را حذف کنیم.

۵.۳. اجرای تست های خودکار و راستی آزمایی جامع



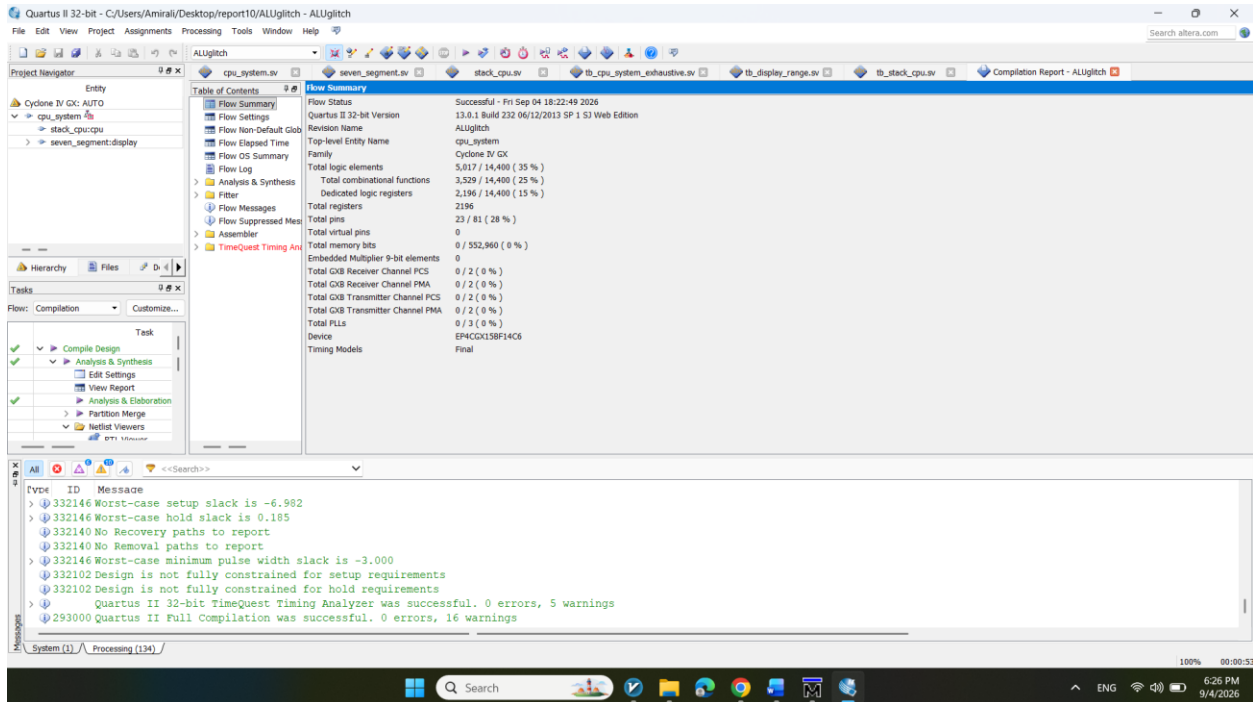
```
Transcript
File Edit View Bookmarks Window Help

# Break in Module tb_display_range at C:/Users/Amirali/Desktop/DSD10/tb_display_range.v line 43
VSDM10> quit -sim
# Compile of cpu_system.v was successful.
# Compile of seven_segment.v was successful.
# Compile of stack_cpu.v was successful.
# Compile of tb_cpu_system_exhaustive.v was successful.
# Compile of tb_display_range.v was successful.
# Compile of tb_stack_cpu.v was successful.
# 6 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.tb_display_range
# vsim -gui work.tb_display_range
# Loading sv_std.std
# Loading work.tb_display_range
# Loading work.cpu_system
# Loading work.stack_cpu
# Loading work.seven_segment
# WARNING: No extended dataflow license exists
VSDM11> add wave *
VSDM11> run -all
# PASS: output-range flag verified for all 256 signed inputs
# ** Note: $finish : C:/Users/Amirali/Desktop/DSD10/tb_display_range.v(41)
# Time: 281 ns Iteration: 0 Instance: /tb_display_range
# 1
# Break in Module tb_display_range at C:/Users/Amirali/Desktop/DSD10/tb_display_range.v line 41
VSDM12> quit -sim
# Compile of cpu_system.v was successful.
# Compile of seven_segment.v was successful.
# Compile of stack_cpu.v was successful.
# Compile of tb_cpu_system_exhaustive.v was successful.
# Compile of tb_display_range.v was successful.
# Compile of tb_stack_cpu.v was successful.
# 6 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.tb_cpu_system_exhaustive
# vsim -gui work.tb_cpu_system_exhaustive
# Loading sv_std.std
# Loading work.tb_cpu_system_exhaustive
# Loading work.stack_cpu
# WARNING: No extended dataflow license exists
VSDM14> run -all
# PASS: all 256 inputs satisfy Y=(X+23)*2-12 mod 256
# ** Note: $finish : C:/Users/Amirali/Desktop/DSD10/tb_cpu_system_exhaustive.v(39)
# Time: 94716 ns Iteration: 0 Instance: /tb_cpu_system_exhaustive
# 1
# Break in Module tb_cpu_system_exhaustive at C:/Users/Amirali/Desktop/DSD10/tb_cpu_system_exhaustive.v line 39
VSDM15>
```

پنجره خروجی متنی در مدل سیم که تاییدیه اجرای بدون خطای تمامی ۲۵۶ حالت ورودی و بررسی پرچم های پردازنده را گزارش می دهد. توجه داشته باشید که این تست بنچ مربوط به cpu_system.v جدید است و صحت همه موارد بررسی شده بدون هیچگونه گلیچ را تایید میکند.

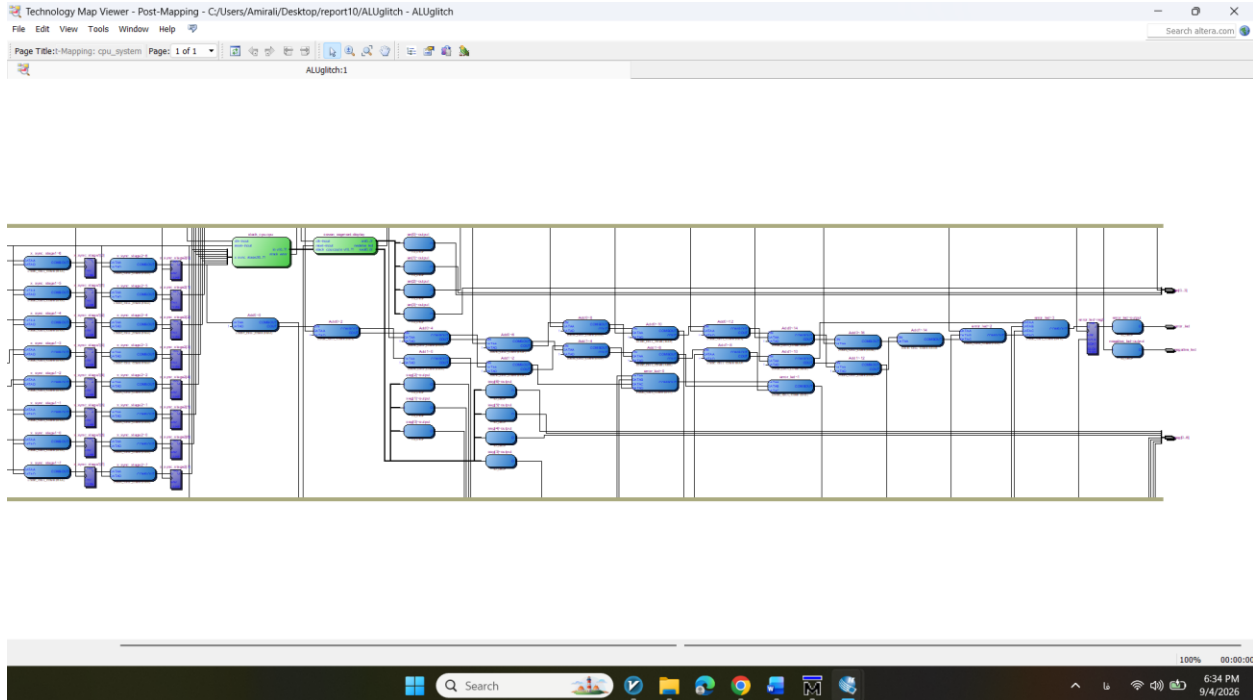
۶. بررسی نتایج سنتز در نرم افزار کوآرتوس

۶.۱. گزارش استفاده از منابع تراشه



گزارش نهایی به کارگیری منابع سخت افزاری شامل تعداد فلیپ فلاپ ها، المان های منطقی و خطوط ورودی و خروجی استخراج شده از گزارش کوآرتوس را مشاهده می کنید. مدار در مجموع ۳۵ درصد از کل ظرفیت منطقی تراشه را پر کرده است (۵۰۱۷ المان منطقی از کل ۱۴۴۰۰ المان موجود) که حدود ۲۵ درصد از ظرفیت تراشه برای مدار های ترکیبی استفاده شده است.

۶.۲. تحلیل مدار در نمای نگاشت فناوری (Technology Map Viewer)



دیگرام نگاشت سخت افزاری که نشان دهنده مسدود شدن مسیرهای تاخیردار ترکیبی توسط فلیپ فلاپ های خروجی و کاهش احتمال بروز ناپایداری است.

این دیگرام ساختار فیزیکی پیاده سازی شده بر روی تراشه را نشان می دهد:

۱. در سمت چپ تصویر، دو ستون عمودی فلیپ فلاپ های همگام ساز ورودی قرار دارند که سیگنال کلیدها را برای حذف پدیده فرا پایداری و ناهماهنگی زمانی فیلتر می کنند.

۲. دو بلوک سبز رنگ بالایی به ترتیب مازول پردازنده پشته ای (**stack_cpu**) و راه انداز نمایشگر (**seven_segment**) هستند.

۳. زنجیره المان های منطقی میانی، محاسبات بررسی سرریز بازه را انجام می دهند که در نهایت به فلیپ فلاپ خروجی متصل شده اند تا از انتقال هرگونه گلیچ ترکیبی به پایه هشدار خطا (**error_led**) و پایه های خروجی تراشه جلوگیری شود.

۷. نتیجه گیری

در این پروژه، پردازنده تک پشته ای هشت بیتی با موفقیت طراحی، تحلیل و مقاوم سازی شد. اثرات تاخیرهای واقعی سخت افزار مانند تاخیر انتشار، زمان های برپایی و نگهداری، و پالس های سوزنی ناخواسته مورد بررسی قرار گرفتند. با به کارگیری تکنیک هایی همچون فیلتر کردن ورودی ها با همگام ساز دو مرحله ای و رجیستر کردن سیگنال های پر تاخیر، سامانه ای بسیار پایدار حاصل شد که خروجی رابطه جبری مورد نظر را بدون پرش، چشمک زدن ناخواسته یا خطای داخلی محاسبه می نماید.